Little Olive Analytics Demo

Project Overview and User Guide

This guide explains what the project does, why it is useful, how to run and use it, how data moves through the system, and what its limitations are. It is written for a project reviewer, client, or interview discussion.

1. What is this project?

Little Olive is a small restaurant website connected to an analytics system. The website lets a visitor browse a sample menu, interact with menu items, place a demonstration pickup order, and open an analytics dashboard.

Behind the website, a reusable browser tracking SDK sends page views and user interaction events to a Laravel API. Laravel validates and stores those events in SQLite. When an order is created, Laravel calculates the price on the server, stores the order, and records a trusted `order_created` analytics event. The dashboard queries the stored events and presents traffic and activity metrics for a chosen date range.

In short, the project demonstrates this complete flow:

```
Visitor uses restaurant website
        -> React records browser activity
        -> Laravel validates and stores events
        -> visitor submits a demo order
        -> Laravel saves order and server event together
        -> dashboard summarizes activity
```

2. What is it useful for?

The project demonstrates how a product team could answer basic questions about a website:

- How many page views happened during a selected period?
- How many distinct anonymous browser IDs viewed a page?
- Which pages were viewed most often?
- How did total event activity change each day?
- Which kinds of activity were recorded?
- Was a server-confirmed business action, such as an order, recorded as an event?

It is also a working example of how to connect a browser tracking SDK, a backend API, durable storage, business logic, and a reporting UI.

3. What a user can do

Restaurant website

- Browse the sample restaurant menu.
- Select a menu item and quantity.
- Submit a demonstration pickup order.
- Generate a `menu_item_selected` custom event by selecting a menu item.
- Generate a `page_view` event when visiting the restaurant page.

The menu, prices, and orders are demonstration data. This application does not take payment, send a real order to a restaurant, or arrange a real pickup.

Analytics dashboard

Open **Analytics** in the website navigation to see:

- **Page Views:** recorded `page_view` events in the date range.
- **Unique Visitors:** distinct anonymous visitor IDs with at least one page view in the range.
- **All Events:** every recorded event type, including page views.
- **Event Activity:** daily event totals and daily page-view totals.
- **Popular Pages:** most-viewed page paths and their distinct visitor counts.
- **Event Mix:** counts for the most common event names.
- **Date controls:** choose an inclusive UTC start and end date, up to 90 days.

Page views are events too. Therefore, one page view normally increases both **Page Views** and **All Events** by one. This is expected: All Events includes Page Views.

4. Technology at a glance

Component	Technology	Responsibility
Website and dashboard	React 18	Renders the restaurant experience, order form, and analytics views
Frontend toolchain	Vite 6	Runs the local frontend server and creates the production build
API and application logic	Laravel 12 / PHP 8.2+	Validates requests, prices orders, records events, and builds summaries
Local database	SQLite	Persists the demo’s orders and analytics events
PHP dependency manager	Composer	Installs Laravel and PHP libraries

Component	Technology	Responsibility
JavaScript dependency manager	npm	Installs React, Vite, and frontend libraries

The project uses one SQLite database file with separate `events` and `orders` tables. A separate database server is not required for the local demo.

5. How to run it on Windows

Requirements

- PHP 8.2 or later, with PDO SQLite enabled.
- Composer.
- Node.js 20 or later.
- npm.

Open PowerShell in the project root, for example `M:\kitchenspurs`.

First-time setup

```
npm.cmd install
Set-Location backend
composer.bat install
Copy-Item .env.example .env
php artisan key:generate
php artisan migrate
Set-Location ..
```

`npm.cmd` is used instead of `npm` to avoid PowerShell execution-policy errors caused by `npm.ps1`.

Start the development application

```
npm.cmd run dev
```

This starts the Laravel API on port `8000` and the Vite development server, normally on port `5173`. Open the **Local** URL printed by Vite in the terminal. If port `5173` is already in use, Vite may choose another port, such as `5174`; use the URL it prints.

Keep the terminal open while using the application. Press `Ctrl+C` in that terminal to stop the development servers.

Production-style local preview

Build the frontend:

```
npm.cmd run build
```

Then start the frontend preview and Laravel API in separate terminals:

```
npm.cmd run preview
```

```
npm.cmd start
```

The preview server is for checking the built frontend locally. Actual production deployment should configure a production web server to serve the frontend and forward API requests to Laravel.

6. How to use the website

1. Start both servers with `npm.cmd run dev`.
2. Open the Vite URL displayed in the terminal.
3. Visit the restaurant page. A `page_view` event is sent automatically.
4. Select different menu items. Each selection sends a `menu_item_selected` event.
5. Submit a sample order. Laravel calculates the total using its own menu prices, saves the order, and records an `order_created` server event.
6. Open **Analytics** in the navigation.
7. Choose a date range to inspect the dashboard. The date range is inclusive and limited to 90 days.
8. Return to the restaurant page and interact again; revisit the dashboard to see the added events.

Events appear on the dashboard only when their UTC `occurred_at` timestamp falls within the selected date range.

7. How the event tracking works

Browser events

The reusable browser SDK in `src/analytics.js` provides `track()`, `pageView()`, and `event()` methods. It creates an anonymous visitor ID in browser `localStorage` and a session ID in `sessionStorage`. It sends each event to `POST /api/events`.

Example event names:

- `page_view` — a page was viewed.
- `menu_item_selected` — a visitor selected a menu item.

- `order_created` — an order was created; this is recorded by the server, not trusted from a browser.

Browser IDs are anonymous and pseudonymous. They are not login identities. Clearing browser storage, using a private window, or switching browsers/devices can create a different visitor ID.

Server business event

Laravel's reusable `ServerEventTracker` service records trusted backend events. In the order flow, Laravel saves the order row and its `order_created` event inside the same database transaction. If either write fails, the transaction rolls both writes back. The browser cannot choose the event's trusted `server` source.

Duplicate handling

Every event has an `event_id`. The `events` table uses it as its primary key. If a caller retries the same event with the same ID, the API reports it as a duplicate instead of creating another row.

8. API overview

The application exposes JSON endpoints under `/api`:

Method	Endpoint	Purpose
`POST`	`/api/events`	Validate and store a batch of 1–50 browser or authorized server events
`GET`	`/api/analytics/summary?from=YYYY-MM-DD&to=YYYY-MM-DD`	Return metrics and ranked summaries for a UTC date range
`GET`	`/api/menu`	Return the server-owned demo menu and prices
`POST`	`/api/orders`	Validate and create a server-priced demonstration order

The Vite development server forwards `/api` calls to Laravel at `http://127.0.0.1:8000`. If a separately hosted frontend calls Laravel directly from a different origin, configure `CORS_ALLOWED_ORIGINS` in `backend/.env`. Set `VITE_API_BASE_URL` before building the frontend when the API is hosted at a different base URL.

`ANALYTICS_SERVER_KEY` is a backend secret for trusted server submissions to the ingestion endpoint. Keep it in the Laravel environment configuration; never embed it in React or send it to a browser.

9. Where data is stored

The default local database file is:

```
backend/database/database.sqlite
```

`events` table

Stores the event ID, event name, anonymous visitor and session IDs, source (`browser` or `server`), optional page path, JSON properties, occurrence time, and receipt time. Indexes support common time-range and event/page queries.

`orders` table

Stores the generated order ID, menu item, quantity, server-calculated total in integer cents, visitor/session IDs, and creation time.

The application uses database migrations to create its tables. SQLite is convenient for a small, single-process demonstration because it keeps the setup simple. It is not presented as a high-volume or multi-instance production storage solution.

10. Validation and reliability

- Event batches contain between 1 and 50 events.
- The event ingestion route has a per-IP request throttle.
- Event names and IDs follow constrained formats.
- Event properties are limited to 4 KB.
- Event timestamps must be within the last 30 days and no more than 5 minutes in the future.
- Page paths must be relative paths beginning with `/`.
- Browser submissions of `order_created` are rejected.
- A valid `X-Analytics-Key` identifies a trusted server submission to the ingestion API.
- Dashboard ranges cannot exceed 90 days and are inclusive UTC calendar dates.
- Invalid API input returns a JSON validation error rather than being stored.
- Order and event writes use a database transaction to remain consistent.

11. Tests and build

Run the Laravel API and feature tests from the project root:

```
npm.cmd test
```

Build the production frontend bundle:

```
npm.cmd run build
```

The feature tests cover event validation and deduplication, batch atomicity, summary aggregation and empty days, date validation, server-side order pricing and event recording, and menu output.

12. What this project does not do

- It does not accept real payments or submit real restaurant orders.
- It does not authenticate visitors or link identities across devices.
- It does not guarantee exactly-once page views. Reloads or a newly triggered page-view event can be recorded as another view; idempotency prevents duplicate delivery only when the same event ID is retried.
- Public browser events can be fabricated. The server key protects trusted server event submissions; it should never be exposed in the browser.
- It does not include user consent management, event retention/deletion workflows, bot filtering, an event-processing queue, or a multi-region analytics pipeline.
- SQLite is a local demo choice. Higher-volume or multi-instance deployments would need a shared production database, shared rate limiting, monitoring, and a deliberate event delivery strategy.

13. Project walkthrough for a reviewer

When demonstrating the project:

1. Start the local application and show the restaurant page.
2. Open browser developer tools' Network panel and filter requests by `events`.
3. Reload the page and point out the browser's `page_view` request.
4. Select a menu item and point out the custom event.
5. Submit an order and show the API's server-calculated total.
6. Open the dashboard and show page views, unique visitors, event activity, popular pages, and event mix.
7. Change the date range to show that the summary is filtered.
8. Explain that the order and its `order_created` event are saved together in one database transaction.

One page view normally increases **Page Views** and **All Events** together because page views are included in the total event count.

14. Short project explanation

> "This is a restaurant demo connected to a small analytics platform. React provides the restaurant UI, a reusable browser SDK, and the reporting dashboard. Laravel exposes JSON APIs, validates and stores events, and handles server-owned order pricing. SQLite stores events and orders in separate tables in one local database file.
>
> Browser page views and interactions go through the event ingestion API. When an order is submitted, Laravel calculates the total and saves both the order and a trusted `order_created` event in one database transaction. The dashboard queries those events to show page views, distinct anonymous visitors, popular pages, daily trends, and event mix for a selected date range.

> The design favors a simple, testable local setup. SQLite and synchronous tracking fit this demonstration, while a larger production deployment would require a shared database, operational monitoring, and a durable event delivery strategy.”

15. Code map

File	Responsibility
`src/main.jsx`	Restaurant page, order form, and analytics dashboard
`src/analytics.js`	Reusable browser analytics SDK
`src/api.js`	Frontend API URL helper
`vite.config.js`	Development API proxy
`backend/routes/api.php`	Laravel JSON API route definitions
`backend/app/Http/Controllers/Api/EventController.php`	Event validation, source assignment, deduplication, and ingestion
`backend/app/Http/Controllers/Api/AnalyticsSummaryController.php`	Date-filtered dashboard queries
`backend/app/Http/Controllers/Api/OrderController.php`	Server-priced demo order and transaction
`backend/app/Services/Analytics/ServerEventTracker.php`	Reusable trusted server-side event recorder
`backend/database/migrations/2026_10_14_000000_create_analytics_tables.php`	`events` and `orders` table definitions
`backend/tests/Feature/AnalyticsApiTest.php`	Laravel feature tests for the main API behavior
`README.md`	Short project setup and operating notes